

Cours 1: Introduction à SAS

Luis Alberto Gomez

Master 1 SE et DS2E

Année 2025 – 2026

Plan de la séance

- 1 Objectifs du cours
- 2 Qu'est-ce que SAS ?
- 3 Les fenêtres de SAS
- 4 Structure d'un programme SAS
- 5 Comment exécuter du code
- 6 Les règles d'écriture
- 7 Les commentaires
- 8 Utilisation des librairies
- 9 La documentation SAS
- 10 Erreurs les plus fréquentes

À la fin de cette séance vous serez capables de :

- Comprendre l'environnement SAS
- Comprendre la structure d'un programme SAS
- Utiliser les deux types d'étapes : **DATA step** et **PROC step**
- Exécuter un programme
- Utiliser des commentaires
- Utiliser des librairies
- Utiliser la documentation SAS
- Identifier les erreurs fréquentes

Qu'est-ce que SAS ?

SAS (**Statistical Analysis System**) est un logiciel utilisé pour :

- la gestion de bases de données
- l'analyse statistique
- l'économétrie
- la visualisation de données
- la production de rapports

Il est très utilisé dans :

- la finance
- la santé
- les statistiques publiques
- la recherche académique

Les fenêtres de SAS

Dans l'environnement SAS on trouve plusieurs fenêtres principales :

Editor

Fenêtre où l'on écrit le code SAS.

Log

Affiche les messages d'exécution, erreurs et avertissements.

Output

Contient les résultats produits par SAS.

Results

Permet de naviguer entre les différents résultats générés.

Structure d'un programme SAS

Un programme SAS est composé de deux types d'étapes :

DATA step

Permet de créer ou modifier des bases de données.

PROC step

Permet d'effectuer des analyses statistiques ou des opérations sur les données.

Structure des steps

DATA step

Permet de créer ou modifier un dataset.

```
data nom_dataset;  
    instructions;  
run;
```

PROC step

Permet d'analyser ou traiter un dataset.

```
proc nom_procedure data=dataset;  
    instructions;  
run;
```

Exemple : Création d'un dataset

Programme SAS

```
data exemple;  
    input age revenu;  
    datalines;  
25 2000  
30 2500  
40 3000  
;  
run;
```

Ce que fait ce programme

- crée un dataset appelé **exemple**
- définit deux variables :
 - **age**
 - **revenu**
- saisit trois observations

Exemple : Analyse statistique

Programme SAS

```
proc means data=exemple;  
run;
```

Ce que fait ce programme

Calcule les statistiques
descriptives des variables
numériques du dataset **exemple** :

- moyenne
- écart-type
- minimum
- maximum

Comment exécuter du code

Pour exécuter un programme SAS :

- 1 écrire le code dans la fenêtre **Editor**
- 2 sélectionner le code
- 3 cliquer sur **Run**

Raccourci clavier fréquent : *F3*

Après exécution :

- vérifier le **Log**
- consulter les résultats dans **Output**

Les règles d'écriture (Boundaries)

Les programmes SAS suivent quelques règles importantes :

- Un programme commence par **DATA** ou **PROC**
- Une étape se termine par **run ;**
- Chaque instruction se termine par un **point-virgule ;**

Remarque importante

NB : SAS ne distingue pas les majuscules et les minuscules.

À quoi servent les commentaires ?

Les commentaires servent à expliquer le code et ne sont pas exécutés par SAS.

Commentaire simple

```
* Ceci est un commentaire ;
```

Commentaire sur plusieurs lignes

```
/* Ceci est un commentaire  
sur plusieurs lignes */
```

Utilisation des librairies

À quoi servent les librairies ?

Les librairies permettent d'organiser et de stocker les datasets. **SAS en ligne utilise des chemins virtuels ou des librairies prédéfinies.** Il existe déjà des librairies SAS contenant de nombreux jeux de données prêts à l'emploi.

Créer une librairie (en local)

```
libname ma_lib "/home/mon_utilisateur/sasdata";
```

Créer un dataset dans une librairie

```
data ma_lib.exemple;  
    set exemple;  
run;
```

Le dataset sera sauvegardé dans la librairie `ma_lib`.

La documentation officielle de SAS est très complète.

On peut y trouver :

- la description des procédures
- les options disponibles
- des exemples de code

Documentation en ligne :

<https://documentation.sas.com>

Conseil : toujours vérifier la documentation d'une procédure.

Erreurs les plus fréquentes

- Oublier un ;
- Utiliser une variable qui n'existe pas
- Mauvais chemin de librairie
- Ne pas vérifier le **Log**

Toujours vérifier :

- les **ERROR**
- les **WARNING**

dans la fenêtre **Log**.